LLM-Generated Python Type Annotations: Correctness, Precision, and Code Preservation

ANONYMOUS AUTHOR(S)

Python type annotations improve reliability, maintainability, and tool support, but retrofitting them into existing code remains labor-intensive. Large language models (LLMs) offer a promising approach to automate annotation generation; however, their practical effectiveness depends not only on whether the generated code passes a type checker, but also on annotation precision and preservation of the original program. We present an empirical study of LLM-based Python type annotation generation on the ManyTypes4Py benchmark. We evaluate multiple LLMs under alternative prompting and output settings, including strict and non-strict annotation-only prompts, source-file and stub-file generation, dependency-aware context, and one-shot versus mypy-guided repair.

Our evaluation considers three complementary dimensions: type-checking correctness, measured by mypy success; annotation precision, measured by the use of `Any`; and code preservation, measured by changes beyond annotations and necessary typing imports. We further compare two dependency-context settings: a structure-only internal context that models early-stage type migration, and a type-enriched internal context that models a setting in which neighboring project modules already provide usable type information.

Our results show that type-checking success alone is insufficient for assessing LLM-generated annotations. Type-checker-guided repair can substantially improve mypy success, but may reduce precision or lead models to modify runtime code despite annotation-only instructions. Strict prompts reduce unintended edits but do not eliminate them, while stub generation provides a safer output modality by separating annotation artifacts from executable source. These findings characterize the tradeoffs among correctness, precision, context, and code preservation, and provide guidance for designing practical LLM-assisted Python type-migration workflows.

Type annotations improve program reliability, readability, and tooling in Python, yet manually adding them remains labor-intensive. Large language models (LLMs) offer the potential to automate this task, but their effectiveness in correctness and precision is still unclear. This paper presents a systematic empirical study of seven representative LLMs (GPT-3.5, GPT-4o, O1-mini, O3-mini, GPT-5, Claude 3.7, and Deepseek) on the ManyTypes4Py benchmark. Our study investigates a few important problems: including the stability of LLMs in different calls and how feedback from the mypy type checker affects the effectiveness of generating type definitions. Our analysis covers two complementary dimensions: correctness (type-check success) and precision (extent of `Any` usage). We also study tricky interactions between these two dimensions. Study results show that individual models achieve up to 61.4% compilation success, while ensemble strategies improve coverage beyond any single model. While LLMs are very effective in fixing type errors they made, in presence of error messages from type checkers, they often change the program structure rather than change only the type annotations, even when they were directed not to do so.

CCS Concepts: • **Do Not Use This Code → Generate the Correct Terms for Your Paper**; *Generate the Correct Terms for Your Paper*; Generate the Correct Terms for Your Paper; Generate the Correct Terms for Your Paper;

Additional Key Words and Phrases: Do, Not, Use, This, Code, Put, the, Correct, Terms, for, Your, Paper

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2026 Association for Computing Machinery.
XXXX-XXXX/2026/9-ART $15.00
https://doi.org/10.1145/nnnnnnn.nnnnnnn

ACM Reference Format:
Anonymous Author(s). 2026. LLM-Generated Python Type Annotations: Correctness, Precision, and Code Preservation. 1, 1 (September 2026), 20 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 INTRODUCTION

Type specifications are a simple yet effective mechanism for catching programming errors [27, 35]. There are many other benefits. For example, type annotations help with program maintenance, by making programming intention explicit. They facilitate the development of many useful programming assistants, such as code completion, refactoring, and many others [45, 54]. Static typing, and thus type specifications, have conventionally been unique to static languages, but the desire to integrate static type specifications to dynamic languages has a long history, dates back to the 1990s [6, 8, 18, 41].

While numerous approaches have been developed for inferring type specifications for static languages [27, 35], type inference of dynamic languages is much more challenging [13, 19, 22]. Type inference is feasible when each expression can be inferred a most general type (often called principal type) [11], but this does not hold for dynamic languages. For example, no principal type can be assigned to the operation + in Python.

Machine learning, which learns knowledge from statistic patterns of existing data [5], has emerged as an important method for developing type inference for Python. Several tools have been developed using this idea [32, 33, 48]. However, an important restriction of such approaches is the limited vocabulary problem, making the prediction of user-defined types difficult. Thanks to the transformer architecture [39] and the attention mechanism [46], large language models (LLMs) possess strong context awareness capabilities [10], making them appealing for inferring Python type annotations.

However, the capabilities of generating Python type annotations with LLMs are largely unknown. Although there have been several initial studies [3, 21, 43], many questions remain underinvestigated in both the correctness and precision perspectives, which we detail below.

Correctness The correctness perspective concerns consistency of added Python type annotations between parameters, arguments, assignments, and other subexpressions. Type correctness is important because otherwise the type annotations can be misleading. A main goal of adding type annotations is to ensure their correctness. The existing studies did not fully scrutinize LLMs' capabilities of generating consistent type annotations. For example, the study by Jiang et al. [21] used the number of mypy compilation errors for individual repositories as the main metric for measuring the consistency of type annotations generated by LLMs. Vagin et al. [43] compared the accuracy of PyType [40] and TypeT5 [48] for inferring Python type annotations. They did not investigate correctness at the file level, without reporting type-check success ratios for files. Bharti et al. [3] did report file-level success ratios, but only present summary numbers.

Precision The main theory of integrating static typing and type annotations in Python, and dynamic languages in general, is gradual typing [37, 38, 42] or optional typing. In that theory, a single parameter or variable can be annotated with a fully static type (such as `List[str]`), a dynamic type (written as `Any` in Python annotation), or a partially static type (such as `List[Any]`). When a parameter is annotated with an `Any`, the type checking of expressions using that parameter is turned off. Specifically, a variable of type `Any` can be assigned with an expression of any possible type, it can be assigned to any variable, it can be an argument to any function, and it can be used as a function with any number of arguments of any types.

Two types can be related through the *precision* relation [37], which specifies whether a type is more specific than the other. For example, `List[str]` is more precise than `List[Any]`, which is

in turn more precise than `Any`. Due to the permissiveness of `Any`, its use raises many interesting questions. For example, how often do LLMs use `Any`, do files that use more `Anys` are more likely to pass type checking, and do LLM added annotations have higher ratios of `Anys` than human annotations? Previous studies did not cover any questions related to `Any`.

Research questions In this paper, investigate the following questions that are skipped in previous studies in detail. First, does invoking the same LLM at different times give different results for the same file? The answer to this question indicates the usefulness of repeatedly calling an LLM to generate better results. This question also allows us to investigate the interactions between correctness and precision in typed programs. Second, how do LLM annotations compare and complement human annotations? The answer indicates when and where LLMs can still improve over human annotations and vice versa. Earlier work on type migration suggested that adding static types to dynamic programs (type migrations) are often done in multiple passes [7, 25, 34]. Third, is the inference of type annotations based on memorization [49] (since LLMs may have trained on the benchmark we used for evaluation and have already seen the type annotations) or on learning such that they are indeed context-aware and can reason? The answer to this question indicates the generalizability of this work. It also implies what performance developers using LLMs could expect.

For each of the first two questions, we study both the correctness (whether the annotated pass static type checking) and the precision (how precise the type annotations are).

Main contributions and observations This paper makes the following contributions. As a study paper, we also list important observations from each question and dimension studied.

(1) We investigate the correctness and precision of generating Python type annotations for the ManyTypes4Py dataset, which consists of an extensive number of smaller projects. We consider seven LLMs, including both older models (GPT-3.5, GPT-4o), newer ones (O1-mini, O3-mini, Deepseek, and Claude Sonnet 3.7), and GPT-5, one of the latest models.
(2) For all models except GPT-5, we run two rounds of each of the programs tested. The overall success ratio (including being successfully processed and passing type checking) of different LLMs ranges from about 13% to 61% (Section 4.1). Models are relatively unstable to generate correct type annotations (Section 4.1), but are more stable to generate similarly precise annotations (Section 4.2). We investigate the impact of precision on correctness and find that, except for syntactic errors, more precise type annotations cause more than 50% of type errors (Section 4.3).
(3) We compare LLM generated annotations and human annotations. We find that human annotations achieve much higher correctness ratio. For files with large parameter counts, the gap is about 50% for best performing LLMs (Section **??**). On the other hand, LLM generated type annotations have much higher coverage than human annotations (Section **??**).
(4) We investigate the impact of existing type annotations on LLMs behaviors. We observed that the amount of existing type annotations radically affects both the correctness (Section **??**) and precision (Section **??**) outcome.
(5) Throughout the study process, we generated numerous Python scripts for processing, analyzing, and interacting with LLMs and extensive datasets from analysis. We will release them to facilitate future research.

The rest of the paper is organized as follows. Section 2 presents an example to demonstrate the correctness and precision aspects of adding type annotations. Section 3 presents the experiment setup. Section 6 answers the question of whether LLMs learn or memorize type annotations. Section 8 discusses threats to validity, Section 9 relates this paper to the literature, and Section 10 concludes.

```
def f(x,y):
    return x if x in y else 1

v1,v2,v3 = f(3,4),f(3,[4]),f(3,3:'a')

# f0 has inconsistent types
def f0(x:int,y:int) -> int:
    return x if x in y else 1

def f1(x:Any,y:List[int]) -> int:
    return x if x in y else 1

def f2(x:int,y:List[int]) -> int:
    return x if x in y else 1
```

```
T, U = TypeVar('T'), TypeVar('U')

def f3(x:T, y:Any) -> Any:
    return x if x in y else 1

def f4(x:T,y:Iterable[U]) -> Union[T,int]:
    return x if x in y else 1
```

	Runtime	f_1	f_2	f_3	f_4
v1	Error	Rejected	Rejected	Accepted	Rejected
v2	Normal	Accepted	Accepted	Accepted	Accepted
v3	Normal	Rejected	Rejected	Accepted	Accepted

Fig. 1. Untyped function f and its typed variants (f_1–f_4). The table reports runtime correctness and static type-checking outcomes.

2 CORRECTNESS AND PRECISION THROUGH AN EXAMPLE

We use the function f and the three values (v1 through v3) that denote the results of calling f in Figure 1 to illustrate correctness and precision. The types Union, Optional, and Any are defined in Python typing module and are self-explanatory. T and U are two named type variables.

Correctness makes sure that all added type annotations are consistent based on the program structure and the rules for assigning types to subexpressions and expressions. For example, the type annotations for f_0 are incorrect because the type for the parameter y violates its usage in the function body. Specifically, the body needs y to be iterable, but the type int does not support that.

The precision aspect investigates the concreteness of type annotations. In reminiscent of the dynamic type in gradual typing theory, a parameter assigned Any means that the static type checking of the uses of the parameter should be skipped. As a result, a single function could be given many possible types. For example, f_1 through f_4 represent a few possibilities how f may be annotated.

The function f_1 is statically well typed because the uses of Any are not statically type checked. Specifically, since x has type Any, the value returned may have the type Any or int. As both types are consistent with int, int can be used as the return type of f_1. Functions f_2 through f_4 are also statically well typed.

A precise type annotation aims to reject all programs that have runtime errors (type system soundness) and accept as many normal programs (without runtime errors) as possible (completeness). As a result, some type annotation is more preferable than others based on the use contexts. The table in Figure 1 presents the results of typing v1 through v3 with f_1 through f_4. The problem with f_3 is that it accepts v1 even though it will encounter a runtime error when executed. The problem with f_1 and f_2 is that they will reject v3 even though it incurs no runtime errors. Thus, the type annotation of f_4 is most preferable for typing v1 through v3.

In general, both aspects are important, which inspires us to evaluate them for each of the first two questions we investigate.

3 EXPERIMENT SETUP

3.1 Datasets

To evaluate the ability of large language models (LLMs) to generate type annotations in Python, we selected datasets that provide both breadth across domains and depth in annotation diversity.

Table 1. Summary of ManyTypes4Py Benchmark (50 Projects, 2,555 Python Files)

Metric	Total	Min	Max	Avg
Lines of Code	1,685,665	31	13,867	659.75
Functions	76,104	5	724	29.79
Parameters	173,828	5	1,428	68.03
Parameters (user annotated)	73,299	0	906	28.69
Return Annotations	40,256	0	431	15.76

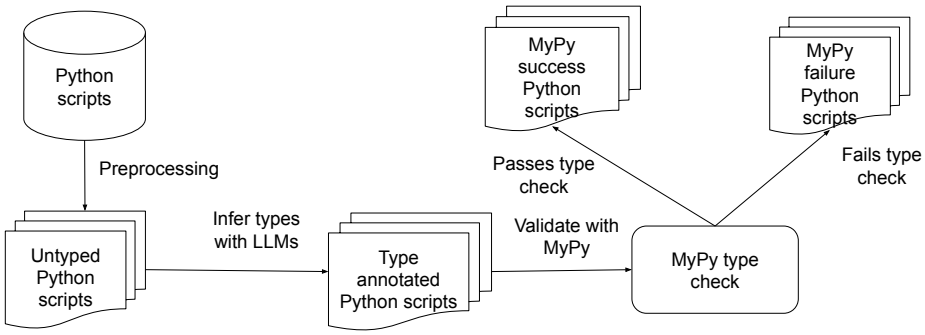


Fig. 2. Overall architecture of the annotation evaluation pipeline. Raw Python scripts are stripped of type annotations and passed to LLMs for type inference. Candidate annotations are validated with mypy; files that pass are retained for further study, while failures are analyzed to characterize error patterns.

Selection Criteria. Datasets were chosen according to three criteria: (i) availability of realistic, non-trivial Python code from actively maintained projects, (ii) diversity of domains (e.g., scientific computing, web development, utilities) to avoid bias toward a single coding style, and (iii) presence of partial or complete type annotations to enable ground-truth comparison.

This study focuses on the ManyTypes4Py dataset. The ManyTypes4Py dataset is the de facto standard for research on Python type inference. From this benchmark, we selected the 50 largest and most representative projects. To ensure sufficient complexity, we further retained only those source files containing at least two functions and no fewer than five formal parameters, resulting in a curated corpus of 2555 Python files. Table presents basic statistics of the dataset.

3.2 Pipeline

Figure 2 illustrates our evaluation pipeline. The process begins with a collection of Python scripts, from which we generate fully untyped versions by removing all existing annotations. Although many projects are originally untyped, stripping annotations from partially annotated files ensures that models infer types independently rather than reusing preexisting hints.

The untyped scripts are then passed to large language models (LLMs) through their public APIs to produce type-annotated versions. We evaluated seven representative models: GPT-3.5, GPT-4o, O1-mini, O3-mini, GPT-5, Deepseek, and Claude Sonnet 3.7. This selection covers both proprietary and open-weight models. We include GPT-3.5 and GPT-4o as widely used general-purpose models with strong performance on code-related tasks. The O1-mini and O3-mini variants were chosen for their cost-effectiveness compared to their larger counterparts (O1 and O3), making them realistic

candidates for scalable use. Deepseek was selected as a state-of-the-art open-weight model, enabling reproducibility for the research community. Finally, Claude 3.7 is widely adopted for software engineering tasks and thus provides an important point of comparison for coding-related annotation performance.

Since no single prompting strategy is universally effective, we experimented with multiple prompt formulations per model. Early designs sometimes produced extraneous or malformed code, necessitating iterative refinement. Through systematic exploration, we converged on stable prompts that consistently yielded valid type annotations. To account for model-specific context length limits, all prompts were tokenized and adjusted to remain within maximum input size.

Each file was independently processed, and annotation latency was recorded for later analysis. We tried to give whole projects to LLMs to provide more context, but none of the models was able to finish a single project. We thus focused on the file level. A retry mechanism handled rate limits and transient API failures. Model outputs were post-processed to extract valid Python code, which was organized into a structured corpus. Only files free from critical syntax errors were retained for downstream evaluation.

The annotated files were then validated using `mypy`, the de facto Python type checker. This stage determined whether the generated annotations were syntactically valid and type-consistent. The files that passed formed the successful set, while those that failed were categorized for further analysis. We then examined both sets in detail, studying error types, frequencies, and relationships to annotation complexity.

3.3 Environment

All experiments were conducted on a Windows 11 workstation equipped with 64 GB of RAM and an Intel® Core™ i9-9900K CPU running at 3.60 GHz (8 physical cores, 16 threads). We used Python 3.10 to implement various scripts for processing files, interacting with LLM APIs, extracting type annotations, comparing annotation precisions, etc. Static type checking was performed using MyPy version 1.15.0. We selected MyPy for its robust error reporting, strong community support, and widespread adoption in Python development workflows.

4 PERFORMANCE OVER TIME

This section evaluates how reliably large language models (LLMs) generate type annotations when applied repeatedly to the same corpus. Our analysis focuses on two complementary dimensions: *correctness*, which measures whether LLM-annotated programs compile successfully under static type checking, and *precision*, which captures the extent to which models avoid over-generalizing to permissive types such as `Any`. Evaluating both aspects is essential: correctness determines whether annotations are immediately usable, while precision reflects their long-term utility for error detection, refactoring, and program comprehension [1, 51]. By comparing multiple runs across seven LLMs, we aim to characterize not only model accuracy but also their stability, consistency, and the trade-offs they make between permissiveness and strictness in type inference.

4.1 Correctness

Overview To evaluate the correctness of LLM-generated type annotations, we processed each file from the ManyTypes4Py dataset, which contains 2555 files. In the dataset, 1040 files are rejected by `mypy` even when all of their type annotations are removed. This phenomenon is a direct consequence of incompleteness of type checking, a well-known property of static type checking [35]. We leave out these files for further consideration, giving us a total of 1515 files in our experiments.

Table 2. Evaluation of static type checking for LLM-generated annotations across runs of various models. GPT-5 had one run; all others had two runs. Numbers for different runs are separated by "/".

Model	Unprocessed	LLM-WT	LLM-IT	LLM-TCS (%)	LLM-OVS (%)
GPT-3.5	993 / 993	202 / 199	320 / 323	38.7 / 38.1	13.3 / 13.1
GPT-4o	327 / 303	644 / 640	544 / 572	54.2 / 52.8	42.5 / 42.2
O1-mini	357 / 358	651 / 657	507 / 500	56.2 / 56.8	43.0 / 43.4
O3-mini	144 / 126	916 / 930	455 / 459	66.8 / 67.0	60.5 / 61.4
Deepseek	585 / 596	587 / 580	343 / 339	63.1 / 63.1	38.8 / 38.3
Claude 3.7	185 / 246	802 / 724	528 / 545	60.3 / 57.1	52.9 / 47.8
GPT-5	214	800	501	61.5	52.8
Llama-3.1-8B-Instruct	516	342	657	34.23	22.57

Each file was sent to every LLM, which adds type annotations to parameters and return types. Each program with type annotations is then type-checked with `mypy`. We categorize the outcomes as follows:

- **Unprocessed**: These files suffer from non-type-related `mypy` failures, such as incomplete definitions, missing return statements, and mismatched parentheses or brackets. These problems are caused by LLM processing errors, such as exceeding context window limits.
- **LLM-WT**: Files for which untyped versions passed `mypy` checking and LLM generated type annotations also passed type checking.
- **LLM-IT**: Files for which untyped versions passed `mypy` checking but the annotations generated by the LLM failed type checking.

To measure the quality of LLM generated type annotations, we define two metrics, LLM-TCS and LLM-OVS, which measure the type checking success ratio and the overall success ratio, respectively. They are defined as follows:

$$\begin{aligned} \text{LLM-TCS} &= \frac{\text{LLM-WT}}{\text{LLM-IT} + \text{LLM-WT}} \\ \text{LLM-OVS} &= \frac{\text{LLM-WT}}{\text{LLM-WT} + \text{LLM-IT} + \text{Unproceed}} \end{aligned} \tag{1}$$

To investigate LLMs' performance over different runs, we repeated each experiment twice except for GPT-5, which was not repeated due to its relative late release. Table 2 reports the results.

From Table 2, we observe the following. First, successful file processing remains a challenge for LLMs. For example, the unsuccessful processing ratios range from about 10% (144/1515) for O3-mini to about 60% for GPT-3.5. Notably, as a relatiely recent model, Deepseek still experiences a high unprocessed ratio, close to 40% (596/1515). Second, more recent models achieve higher LLM-TCS ratios than earlier generations. For example, O3-mini reaches 67.0%, Deepseek also performs relatively well at around 63%, and O1-mini and Claude 3.7 occupy a middle range, typically between the mid-50s and mid-60s. In contrast, older models show weaker performance: GPT-3.5 achieves less than 40% for the dataset. Third, the overall success ratios (LLM-OVS) trend that of type checking success ratios (LLM-TCS) with smaller numbers.

Put into the context of type migration, LLM-TCS is the probability a successfully processed file passes type checking. LLM-OVS is the probability that all added type annotations pass type checking, without any interventions required from the user. The low LLM-OVS numbers in Table 2 indicate that, even though substantial progress has been made, heavy human intervention is still needed to add type annotations to Python programs.

Table 3. Run-to-run consistency of LLM-annotated files for the ManyTypes4Py benchmark. Columns two through five report file counts for categories (a)–(e) discussed in the paper. Filled and unfilled circles denote success and failure, respectively.

Model	Unprocessed	●●	●○	○●	○○	Instability Ratio (%)	2nd-↑ ratio
GPT-3.5	905	154	48	45	363	15.2	3.4
GPT-4o	285	597	47	43	543	7.3	4.9
O1-mini	219	499	152	158	487	23.9	18.3
O3-mini	72	809	107	121	406	15.8	20.0
Deepseek	482	491	96	89	357	17.9	9.5
Claude 3.7	178	676	126	48	487	13.0	6.7

Run-to-run comparison Table 2 shows that the results across different runs have relatively stable LLM-TCS values. For example, for most LLMs, the difference is within 2%, except for Claude 3.7. This raises an interesting question, are the files that pass type checking in the first run and the second run almost the same or quite different? The answer can be useful for two reasons. First, since LLM generation is inherently stochastic, run-to-run consistency is critical for judging reliability in practice [9, 51]. Second, the answer indicates whether calling an LLM multiple times to increase the success ratio is a viable idea. If the set is almost the same, then repeating will yield little gain. In contrast, if two sets are quite diverse, then calling an LLM multiple times will likely generate correct type annotations for programs that earlier generations failed.

We categorize the outcomes of running each program in two runs as: (a) unprocessed in both runs, (b) success in both runs, (c) success only in the first run, (d) success only in the second run, or (e) failure in both. We also define the instability ratio, which is the proportion of files that succeeded in only one run ((c)+(d)) over all files that are successfully processed (the sum of (b) through (e)). Finally, we define 2nd ↑ ratio, which is calculated by dividing the count of second-only success ((d)) over files that did not succeed in the first run ((a)+(d)+(e)). Intuitively, this ratio indicates that if a file did not succeed in the first run, what is the probability that it will succeed in the second run.

Table 3 presents the results. Note, the values for the "Unprocessed" column in Table 3 count files failed to process in both runs and are thus smaller than those in Table 2. The result exhibits several interesting phenomena. First, except for Claude 3.7, the numbers of 1st-only success (column four) and those of 2nd-only success (column five) are very close, even though the values themselves are quite large. Interestingly, that means that if an LLM makes unfortunate mistakes for a certain set (in the sense that another run may have succeeded this set) of programs, then it is fortunate enough to get a set (in the sense that another run may have failed this set) of similar size of programs correct.

Second, the table shows that LLMs in general lead to a large instability ratio. Except for GPT-4o, all other models have instability ratios close to or greater than 15%. A high instability ratio means that a file that is success in one run may fail in another run. This has a negative impact on reproducibility. Third, different models have very different 2nd-↑ ratios. The number ranges from 3.5% for GPT-3.5, 9.5% for Deepseek, to 20% for O3-mini. Since this ratio measures the possibility of success upon the first failure, a higher ratio implies greater benefits of retrying.

4.2 Precision

Overview Correctness alone does not capture the *precision* of inferred types. A model can artificially increase its chance of passing `mypy` by over-using the dynamic type `Any`, which suppresses type checking and prevents errors from being detected. Although this strategy can improve short-term compilation success, it diminishes the long-term advantages of static typing, including accurate

Table 4. Usage of Any in LLM-generated annotations. Reported are total slots (excluding `self`s), the number of Any, overall Any ratios, and Any ratios in parameter and return types.

Model	Total	Any Slots	Any ratio	Param Any ratio	Return Any ratio
GPT-3.5	39054 / 41021	7330 / 7306	18.77% / 17.81%	25.78% / 24.51%	6.30% / 6.02%
GPT-4o	62589 / 63499	5552 / 5595	8.87% / 8.81%	13.12% / 12.93%	2.29% / 2.41%
O1-mini	83701 / 83846	6219 / 6145	7.43% / 7.33%	10.61% / 10.38%	2.66% / 2.78%
O3-mini	86288 / 85727	11390 / 10947	13.20% / 12.77%	18.69% / 18.00%	5.04% / 5.01%
Deepseek	60302 / 60016	4756 / 4779	7.89% / 7.96%	11.15% / 11.26%	2.84% / 2.87%
Claude 3.7	66728 / 66014	6064 / 6052	9.09% / 9.17%	13.21% / 13.30%	2.73% / 2.78%
GPT-5	85946	8394	9.77%	14.40%	2.64%

error detection, safe refactoring, and strong program invariants [1, 31, 51]. Measuring reliance on Any is therefore essential for evaluating whether LLM-generated annotations are not only correct but also semantically meaningful. In our analysis, we define Any to include both explicit Any annotations and empty slots (where annotations were absent). In our analysis, we ignore the first parameter of each method, usually named as `self`. There is no need to give a type for this parameter. We compute the Any ratio as follows.

$$\textit{Any ratio} = \frac{\textit{Number of Any slots}}{\textit{Number of total slots}}$$

We present the results of Any usage in Table 4. The table shows results for all files with LLM-generated annotations, regardless of whether they passed mypy checking or not. We also analyzed Any usage for files that passed mypy checking and for those that failed. Since the results are very close and they do not offer new insights, we defer their presentation to the appendix.

From Table 4, we make three observations. First, all models assign type annotations to most of the parameters and return types. For example, the non-Any ratios (1-Any ratio) are always greater than 75%. Second, some notable differences exist in how often LLMs assign Anys. For example, O1-mini and Deepseek use about 10% Any in parameters while O3-mini uses 18% and GPT-3.5 uses 25%. Third, all models use many more Anys in parameters than in return types. For example, 18% of parameters in O3-mini received Anys, and that number dropped to 5% for return types. A likely reason for LLMs to assign Anys to parameters far more often than to return types is that parameters are structurally harder, semantically underspecified, and riskier to guess, whereas return types are usually much easier to infer from the function body.

Run-to-Run Consistency of Any Predictions While Table 4 reported the overall frequency of Any, it did not capture whether models make these predictions consistently across repeated runs. To examine stability, we compare type slots (function parameters and returns) present in both runs of each model and classify them as:

- **Any both**: slots predicted as Any in both runs,
- **Any 1st / Any 2nd**: slots predicted as Any in only one run,
- **Any Neither**: neither of the slots was predicted Any. We further classify this case into four subcases: (a) the first run is more precise (denoted as $\nwarrow$ in Table 5), (b) the second run is more precise (denoted as $\nearrow$), (c) types in two runs are incomparable (denoted as $\leftrightharpoons$), and (d) the two types are exactly the same. The precision relation is standard in gradual typing [38].

To measure the type relations between two runs, we introduced three ratios. The first is $R^{\text{1Any}}_{\text{All}}$, which is the sum of Any 1st and Any 2nd over total slots, measuring the likelihood that the Any

Table 5. Run-to-run comparison of Any predictions and precision relations across models.

Model	Total	Any ...			Any Neither					R^{1Any}_{All}	R^{NEQ}_{NAny}	R^{NEQ}_{All}
		Both	1st	2nd	All	↖	↗	⇋	=			
GPT-3.5	4,444	996	55	72	3,321	1	1	41	3,278	3.09%	1.3	4.3
GPT-4o	22,216	2,034	114	94	19,974	33	34	488	19,419	0.94%	2.8	3.7
O1-mini	17,693	1,049	306	283	16,055	42	60	1,094	14,859	3.34%	7.3	10.4
O3-mini	33,272	3,344	610	538	28,780	145	77	1,784	26,774	3.46%	7.2	10.6
Deepseek	14,127	989	125	179	12,834	19	31	521	12,263	2.15%	4.3	6.3
Claude 3.7	25,507	2,439	18	41	23,009	5	12	175	22,817	0.23%	0.8	1.0

prediction is flipped across two runs. The second is R^{NEQ}_{NAny}, which is the ratio of types not equal over Any neither. The third is R^{NEQ}_{All}, which measures the probability that the types across two runs are different.

We present the result in Table 5. Note that the *Total* column in the table differs from that in Table 4, as it only includes parameter and return slots that are present in both runs of a given model. From Tables 4 and 5, we make the following observations. First, while Anys are quite common in LLM predicted types, their uses are quite stable across different runs. Second, there is no direct correlation between how often LLMs use Anys with how stable Anys are used. For example, Claude 3.7 uses more Anys than O1-mini but the former is much more stable in using Anys. Third, it is quite common for different runs to use different non-Any types, sometimes one is more precise than the other but it is more likely that they cannot be related through the precision relation.

4.3 Interactions between correctness and precision

Table 3 shows that for each LLM, a certain number of files are successfully added with type annotations in one run but not the other. Table 5 further shows that in different runs the same parameter may be assigned different types. Sometimes one is more precise than the other, and other times they do not satisfy the precision relation. Type precision can affect correctness in several ways. For example, in Figure 3, the first run assigns the more precise type `type` to `dtype_cls`. However, since values of `type` do not have the attribute `_checker`, this type annotation causes a type error. Instead, the `Any` type given to `dtype_cls` in the second run removes the type error.

As another example, consider the program in Figure 4, where the variable `value_template` is assigned the type `Template` and `Optional[Template]` in the first and second runs, respectively. While early work on gradual typing [37] does not relate a type `T` and its optional version `Optional[T]`, later work that extends the idea of gradual typing to nulls [24, 30] defined `T` to be more precise than `Optional[T]`. In this example, the parameter `value_template` is assigned to the field `_value_template`. As a result, the reference `self._value_template` in `update` has the same type as the parameter `value_template` in `__init__`. The first run leads to a type error because the `None` variant of `Optional[Template]` does not have the attribute `render_with_possible_json_value`. The second run assigns a more precise type and does not have a type error.

Since correctness and precision are closely related, it is interesting to know how often more or less precise types lead to type errors. For that reason, we classify all files that failed in exactly one run into following reasons: (a) the failure is due to the use of static types for some parameters in one run and is avoided by using `Any` in the other run, (b) the failure is caused by the use of `Optional[T]` in one run and is avided in the other run by using `T`, (c) the failure is because one run uses undefined type, such as using `Callable` without importing it, and is defined in the other run,

```
def _coerce_to_data_and_mask(
    values: Any, dtype: Optional[Any],
    copy: bool, dtype_cls: type,
    default_dtype: Any) -> ...:
    checker = dtype_cls._checker
    mask: Optional[np.ndarray] = None
    ...
```

```
def _coerce_to_data_and_mask(
    values: Any, dtype: Optional[Any],
    copy: bool, dtype_cls: Any,
    default_dtype: Any) -> ...:
    checker = dtype_cls._checker
    mask: Optional[np.ndarray] = None
    ...
```

Fig. 3. An example where the first run assigns a more precise type to dtype_cls, leading to a type error, whereas the second run uses a less precise type that passes type checking. The gray highlight indicates the differing type annotations, and the blue highlight shows the resulting effect.

```
class DweetSensor(SensorEntity):
  def __init__( self,
    ... omitted three parameters
    value_template: Template,
    unit_of_measurement: Optional[str]
  ) -> None:
    self._value_template = value_template
    ...

  def update(self) -> None:
    ...
    self._value_template
        .render_with_possible_json_value(...)
```

```
class DweetSensor(SensorEntity):
  def __init__( self,
    ... omitted three parameters
    value_template: Optional[Template],
    unit_of_measurement: Optional[str],
  ) -> None:
    self._value_template = value_template
    ...

  def update(self) -> None:
    ...
    self._value_template
        .render_with_possible_json_value(...)
```

Fig. 4. An example where the first run assigns a more precise type to value_template and passes type checking, whereas the second run uses a less precise type and results in a type error.

Table 6. Failure classification for files that succeed in one run and fail in the other run. Columns three through seven report counts for reasons (a)–(e).

Model	Total	Any	Optional	TpUndefined	VarUndefined	Syntax
GPT-3.5	93	19	0	15	12	47
GPT-4o	90	43	3	12	10	22
O1-mini	310	126	9	71	32	72
O3-mini	228	150	7	20	26	25
Deepseek	185	60	6	27	19	73
Claude 3.7	174	88	2	33	25	27

(d) the failure is caused by using undefined value names, such as using cast without importing it from typing, (e) the failure is due to syntactic issues, such as mismatching parentheses, missing return statements and other related issues. Such problems occur because the input programs may be too long for LLMs to process.

We present the results in Table 6. The table shows that syntactic error is one of the main reasons for failure for GPT-3.5 and Deepseek. The result also shows that the uses of Any (the third column) help reduce many type errors, including attribute errors, not supported operator types, incorrect number of arguments (arities specified in the Callable type), function and argument type mismatches, and so on. For models that suffer little from syntactic failures, the uses of Any help

reduce about 50% or more type errors. Undefined types and variables are also common reasons for causing failures.

5 DEPENDENCY

Dependency slicing Because our study evaluates file-level annotation generation, the target file may refer to type-relevant definitions outside the file, including project-local base classes, helper functions, constants, validators, decorators, and re-exported APIs. Providing the whole project to the LLM is impractical: in pilot experiments, large prompts frequently led to incomplete outputs or unintended modifications outside the target file. Conversely, providing only the target file creates an artificial context deficit that can encourage hallucinated types or imprecise annotations such as `Any`. We therefore construct, for each target file, a dependency slice: a compact, read-only API summary containing only the external names that the target file statically uses.

We resolve imports using a project-first strategy. Given a repository root and a target file, we derive the file's module and package name from its path. Absolute imports are resolved by mapping dotted module names to candidate source paths, preferring `.pyi` stubs over implementation files and package `__init__` files. If a module cannot be resolved under the repository root, we employ Python's import specification mechanism in the project's analysis environment to locate standard-library and third-party modules. We classify a dependency as internal when its resolved path lies under the repository root, and external otherwise. Relative imports are normalized to absolute module names using the target file's package, rather than by changing the current working directory or modifying the import path.

To identify the subset of imported APIs that are actually needed, we perform lightweight lexical name resolution over the target file's AST. The resolver maintains scoped bindings for imported modules, imported symbols, local assignments, function parameters, class definitions, loop variables, exception variables, and nested imports. This prevents imported names from being incorrectly attributed after they are shadowed locally. For module imports, we resolve attribute chains against the imported module binding. For example, if `cv` is bound to `myproj.helpers.config_validation`, then `cv.string` records a use of symbol string from that module. For unaliased imports such as `import a.b.c`, we reconstruct attribute chains and match the longest imported-module prefix. Star imports are handled by inspecting statically recoverable exports: `__all__` when available, otherwise public top-level names. Ambiguous star-import resolutions are marked unresolved rather than guessed.

The resulting used-symbol map records, for each dependency module, the specific classes, functions, constants, or attributes used by the target file. We convert this map into a stub-like capsule containing the relevant API surface: function signatures, class skeletons, class attributes, constants, and re-export hints. For external modules, existing stubs are preferred when available; otherwise, conservative placeholders are emitted. The capsule is supplied to the LLM as read-only context, while the LLM is constrained to modify only the target file's type annotations and necessary typing imports.

We use a Python orchestration script to create per-repository virtual environments, install declared dependencies when available, invoke the dependency slicer for each target file, and record slice metadata, logs, and failures.

Two settings: We evaluate two dependency-context settings. In the internal-structure setting, project-internal dependencies are summarized without synthesized type annotations: the slice contains import-resolved names, class and function skeletons, parameter names and defaults, constants in source form, and re-export relationships. This setting models early-stage type migration, where the target project itself is largely untyped.

Table 7. Comparison of Any usage in base vs. renamed files (200-file subset). The change in Any ratio remains marginal across LLMs.

Model	Base (%)	Renamed (%)	Δ
GPT-3.5	33.90	33.86	−0.04
O3 Mini	28.14	29.18	+1.04
Deepseek	25.73	25.42	−0.31
Claude 3.7	24.78	25.94	+1.16

In the internal-typed setting, project-internal summaries are enriched with type facts that are already present in the source or conservatively recoverable from local syntax, such as explicit annotations, literal assignments, aliases, constructor calls to resolvable classes, and bounded recursive type dependencies. This setting models a later-stage or optimistic migration scenario in which neighboring internal modules have already been typed.

In both settings, we retain available type information for standard-library and third-party dependencies, since such stubs are normally available to Python type checkers and developers.

5.1 Evaluation aspects

average/median slice size in tokens or LOC
number of modules per slice
fraction of imports resolved as internal, stdlib, third-party, unresolved
fraction of star imports and ambiguous star-import cases
how often .pyi files were used
how often re-export chasing was needed
effect of slice strategy on mypy success and Any usage

6 DO LLMS LEARN OR MEMORIZE?

To examine whether large language models (LLMs) memorize code from the ManyTypes4Py benchmark, we conducted three experiments.

Correctness and precision The first two experiments assess whether the correctness and precision will be significantly different as we perform program transformations. We specifically considered function renaming, with the following process. First, we randomly chose 200 files from the dataset. Second, we replaced function names and corresponding calls of the chosen files to remove any potential surface overlap with the original dataset. Third, we prompted LLMs to annotate types of new files.

In the first experiment, we compare the correctness ratios of the output files of the 200 files with and without transformation. The results are in line with those presented in Table 3.

In the second experiment, we compare the precision. The hypothesis is that, if models relied heavily on memorization from pre-training exposure, the renamed files would yield substantially higher Any usage compared to the original versions, since renaming disrupts lexical cues.

Table 7 summarizes the results, reporting the mean Any ratio for base vs. renamed files. Across models, the observed changes remain below 1% in most cases and never reach a level indicative of systematic memorization.

Together, these two experiments suggest that we cannot attribute their performance to direct memorization of ManyTypes4Py examples. Instead, the findings indicate that LLMs generalize type annotation behavior in a way that is relatively robust to superficial renaming.

Table 8. Semantic type matching results across original and transformed files. Values are percentages of parameters matching human annotations.

LLM	Both (%)	Orig. only	Ren. only	Neither
GPT-3.5	62.42	6.07	4.58	26.93
O3-mini	74.19	4.58	3.66	17.56
Deepseek	80.27	3.66	2.47	13.61
Claude 3.7	79.63	6.76	2.42	11.20

Semantic understanding In the third experiment, we disentangle semantic understanding from potential memorization. For this purpose, we conducted a semantic type matching analysis. The human-annotated ground truth was compared with LLM-generated annotations on both original files and transformed files. Each parameter or return type was classified into four outcomes: *Match both*, *Match original only*, *Match renamed only*, and *Match neither*. Table 8 reports results for GPT-3.5, O3-mini, Deepseek, and Claude 3.7.

The table shows that Deepseek (80.3%) and Claude 3.7(79.6%) achieve the highest *Match both* ratios, followed by O3-Mini (74.2%) and GPT-3.5 (62.4%). Ratios of *Original only* and *Renamed only* remain consistently low (2–7%), showing that renaming functions rarely changes outcomes. Failures are concentrated in the *Match neither* category, ranging from 11–27%, with GPT-3.5 performing the worst. Mismatches often involve complex constructs such as generators and I/O types.

These findings indicate that LLMs are not merely exploiting function names: high *Match both* ratios and the negligible difference between original and renamed inputs suggest genuine semantic reasoning. The low sensitivity to naming conventions further demonstrates promising generalization across projects with diverse coding styles. At the same time, the persistent fraction of *Match neither* cases highlights that complex or less common type signatures remain a notable source of error, pointing to an important direction for model and training improvements.

Overall, these three experiments suggest that models generate type annotations by learning from the code semantics and type constraints of programs rather than memorizing from early trainings.

7 LLM-BASED MYPY REPAIR: EFFECTIVENESS AND FIX PATTERNS

We evaluate the ability of LLMs to repair `mypy` type errors in their own annotated code using an iterative feedback loop. For each file with initial `mypy` errors, the model is re-prompted with the code and error messages, with strict instructions to modify only type annotations, typing imports, and `TYPE_CHECKING` blocks. The loop runs for up to 5 attempts or until `mypy` succeeds.

Repair effectiveness. Table 9 summarizes the repair outcomes across models.

GPT-5 achieves the highest repair rate (99.0%), resolving nearly all files, while DeepSeek lags significantly (75.2%). Claude provides a strong balance, achieving 93.4% with the lowest average runtime per file. Across all models, most files are repaired within two iterations, indicating that a single feedback cycle is typically sufficient. In contrast, unfixed files consistently exhaust all five attempts and incur substantially higher computational cost.

Fix pattern analysis. To understand *how* repairs are achieved, we diff the original LLM-annotated code against the final repaired version for all successfully fixed files. Each change is classified into nine categories using a hybrid AST-level and line-level analysis. Table 10 reports the distribution of changes. :contentReference[oaicite:0]index=0

Table 9. Summary of the LLM mypy repair loop.

	GPT-5	DeepSeek	Claude
Total files	501	339	528
Fixed	496 (99.0%)	255 (75.2%)	493 (93.4%)
Unfixed	5 (1.0%)	84 (24.8%)	35 (6.6%)
Fixed on 1st	89 (17.9%)	19 (7.5%)	39 (7.9%)
Avg. attempts	2.03	2.92	2.46
Median attempts	2	2	2
Avg. time (all)	130.4s	185.7s	112.8s
Total time	18.1h	17.5h	16.5h

Table 10. Distribution of fix patterns across LLMs.

Category	GPT-5		DeepSeek		Claude	
	Cnt	%	Cnt	%	Cnt	%
Param annotation	434	6.1	146	6.6	544	11.3
Return type	349	4.9	57	2.6	320	6.6
Variable re-ann.	197	2.8	33	1.5	43	0.9
Import addition	180	2.5	95	4.3	120	2.5
Import removal	15	0.2	4	0.2	18	0.4
`cast()`	620	8.7	41	1.8	84	1.7
`None`-guard	216	3.0	124	5.6	279	5.8
`type: ignore`	51	0.7	59	2.7	270	5.6
Other code	5046	71.0	1659	74.8	3153	65.3
Total	7108		2218		4831	

Insights. Despite explicit instructions to restrict edits to typing-related constructs, all models predominantly modify runtime code (65–75%), revealing a fundamental limitation in controlling repair locality. The models also exhibit distinct repair strategies. GPT-5 relies heavily on `cast()` (8.7%), suggesting a coercion-based approach. In contrast, DeepSeek and Claude favor defensive narrowing via `None`-guards (5.6–5.8%), while Claude additionally employs `type: ignore` more aggressively (5.6%), indicating a suppression-based strategy. Annotation refinements (parameters and return types) are more prominent in Claude, whereas import-level changes remain relatively minor across all models.

Overall, these results show that high repair success does not imply principled fixes: LLMs frequently achieve correctness by altering program behavior or introducing ad hoc workarounds rather than performing minimal, type-consistent edits.

8 THREATS TO VALIDITY

Our internal validity may be affected by errors in our preprocessing, type-checking pipeline, mypy version, and its command line arguments. To increase internal validity, we verified parsing, normalization, and Python scripts through unit tests and manual inspection. For mypy, we used the same command line arguments as in [3]. To ensure consistency, we used the same mypy version and arguments for all experiments.

Regarding construct validity, the correctness is defined through mypy pass/fail, and type precision may not fully capture semantic nuances. We used mypy because it is most established, has strong community support, and is actively maintained. To reduce the threat of type comparison results, we use the standard definition of type precision from the gradual typing community.

For external validity, our datasets (ManyTypes4Py) cover diverse real-world code but may not represent proprietary or domain-specific Python codebases. Likewise, we evaluate a limited set of LLMs; as model families evolve rapidly, newer versions may behave differently. To mitigate these threats, we chose both old and new models. Also, while some of our findings characterize specific models, datasets, and configurations studied, we try to study the problem from the typing perspective, such as the impact on increased precision on correctness.

9 RELATED WORK

Type inference strategies Developers and researchers have explored multiple strategies for inferring types in Python. The first line of work extends *static analysis* through gradual typing, introduced via PEP 484 [44]. Tools such as Mypy, PyType, PyRight, and Pyre emerged to check these annotations, and empirical evidence shows that type hints both prevent bugs and persist once adopted [55]. Yet, static inference remains limited: analyzers must over-approximate behavior in the face of Python's dynamic features, often defaulting to `Any` when precise inference is infeasible [14, 20, 23], and dynamic tracing approaches (e.g., MonkeyType) still yield only partial coverage. A second line of work applies *machine learning* and deep learning to predict types directly from code. Early statistical models such as Xu et al. [52] captured probabilistic relationships between identifiers and types, while neural approaches like DeepTyper [17] and NL2Type [26] leveraged sequence models and natural language cues. TypeWriter [36] integrated neural predictions with type checking to prune invalid suggestions, and later systems such as Typilus [2], LambdaNet [47], TypeBERT [16], and Type4Py [28] employed graph neural networks or transformers to capture richer dependencies. While these methods significantly improve coverage, they remain constrained by annotated datasets like ManyTypes4Py [29], struggle with rare or user-defined types, and are often evaluated only on snippet-level benchmarks, leaving project-scale correctness and consistency underexplored.

Most recently, *large language models (LLMs)* trained on massive code corpora have enabled open-vocabulary type inference. Unlike earlier approaches limited to fixed type sets, LLMs exploit training data and contextual cues from identifiers, comments, and usage to generate type annotations. Models such as Codex and GPT-4 integrate code and natural language semantics, while specialized variants like TypeT5 [48] and TypeGen [32] fine-tune transformer architectures for producing structured and contextually relevant annotations. This generative capability allows LLMs to propose novel or project-specific types beyond predefined vocabularies.

Empirical studies Bharti et al.[4] present an automated pipeline that leverages LLMs to generate Python type annotations in an iterative *generate–check–repair* loop. Their approach integrates an LLM with the `mypy` type checker, where annotations are generated, validated, and incrementally repaired until type checking succeeds. Using GPT-based models, they report high file-level type-checking consistency (nearly 88% success) and competitive accuracy compared to supervised systems such as Type4Py. In contrast, our work pursues a broader *empirical study* rather than a single repair-driven pipeline. We evaluate seven diverse LLMs on ManyTypes4Py, focusing on both *correctness* (type-checking success) and *precision* (avoidance of over-generalized `Any` annotations). Beyond correctness ratios, we analyze run-to-run stability, specificity loss, complementarity among models, and systematic comparisons with human annotations. Thus, while Bharti et al. demonstrate the feasibility of a repair-enhanced LLM pipeline, our study provides a comprehensive multi-model

evaluation and exposes fundamental trade-offs in correctness, precision, and model diversity that remain invisible in a single-pipeline setting.

Zieliński et al [55] conducted an empirical study of the evolution of Python type annotations in real-world projects. Their work analyzed a large corpus of open-source repositories to investigate how developers adopt, modify, and maintain type hints over time. The study highlighted that once annotations are introduced, they tend to persist, and that type hints correlate with improved maintainability and bug prevention. In contrast, our work does not focus on developer practices or historical adoption trends, but rather on the *capabilities of large language models* to automatically generate type annotations. While Zieliński et al. provide insights into human annotation behavior and ecosystem-level evolution, we evaluate seven LLMs, quantifying both *correctness* and *precision* of generated annotations. Furthermore, we study four unique research questions not covered by their work, thereby extending the scope of empirical research from human practices to automated annotation systems.

Yee and Guha [53] examine whether machine learning models can generate TypeScript types that successfully type check. Their system, *TypeWeaver*, integrates several type prediction models into a migration pipeline that automatically converts JavaScript code to TypeScript and validates the results using the TypeScript compiler. They show that high prediction accuracy does not always translate into type-checking success due to unsound or inconsistent annotations. In contrast, our study focuses on *Python* and evaluates seven LLMs on the ManyTypes4Py benchmark, measuring both *correctness* and *precision*. We further analyze `Any` usage, run-to-run stability, and complementarities across models. While Yee and Guha highlight the gap between accuracy and type-check validity in TypeScript, our work extends this insight to Python, providing a large-scale, multi-model perspective on the reliability of LLM-generated annotations.

Guo et al. [15] examine how far existing Python type inference engines can be pushed to generate usable annotations. Their work evaluates three categories of inference tools, identifies recurrent failure modes in the translation from inferred types to concrete annotations, and proposes an enhancement tool together with a developer study to assess its usefulness. Their analysis is therefore centered on the *inference-to-annotation pipeline*: accuracy of inferred types, translation feasibility, and developer effort. In contrast, our work studies a fundamentally different problem—*the quality of type annotations generated directly by large language models*. We evaluate seven LLMs on the ManyTypes4Py benchmark across multiple dimensions that are unique in our work: *correctness* (mypy success), *precision* (specificity and `Any` usage), run-to-run *stability*, agreement with human annotations, and *model complementarity*.

Dong et al [12] introduced *TypyBench*, a benchmark specifically designed to evaluate LLM-based type inference at the repository scale. Their work emphasizes two key dimensions: *TypeSim*, a semantic similarity metric for measuring alignment between predicted and ground-truth types, and *TypeCheck*, a repository-level validation of whether generated annotations type-check consistently. This provides a valuable foundation for studying large-scale consistency errors in LLM-generated annotations. Our study differs in both scope and methodology. Rather than introducing a new benchmark, we conduct a broad *multi-model empirical evaluation* across two established datasets, from the perspectives of both correctness and precision. In addition, we investigate four research questions not covered in TypyBench. Thus, while Dong et al. provide a new benchmark and evaluation metrics, our work contributes a comprehensive comparative analysis that highlights practical trade-offs in correctness, precision, and model diversity.

Xu et al. [50] examined whether LLM-based type inference for Java code snippets stems from genuine reasoning or simple memorization of training data. Their study, conducted on a relatively small set of Java functions, evaluated precision (not type relation in our sense but type correctness compared to ground truth), recall, and F1 scores, and included experiments to separate memorization

effects from contextual inference. In contrast, our work targets *Python* and pursues a substantially broader empirical scope, investigating four research questions from two unique perspectives on two datasets. Their study revealed that models rely heavily on memorization rather than learning in Java, but our study revealed otherwise.

10 CONCLUSION

This paper systematically studied LLMs capabilities in generating correct and precise type annotations for Python. We considered seven LLMs, used the ManyTypes4Py dataset, and investigated many questions in depth. Our study reveals that LLMs work very well for small programs but struggle for programs with large parameter counts. LLMs often add type annotations to parameters left out by human annotations. A main challenge of current LLMs is generating consistent type annotations with existing ones and leveraging them to generate more precise annotations.

Overall, the study clarifies the current capabilities and boundaries of LLM-based type annotation, provides reproducible evaluation tools, and highlights concrete directions for future explorations.

DATA AVAILABILITY

The data package, including prompts, evaluated programs, LLM outputs, and other scripts, is available in the following link https://anonymous.4open.science/r/Harmonizing_ml_results-EC8C/README.md. We will submit the package for artifact evaluation upon acceptance.

REFERENCES

[1] Miltiadis Allamanis, Earl T Barr, Premkumar Devanbu, and Charles Sutton. 2018. A survey of machine learning for big code and naturalness. *ACM Computing Surveys (CSUR)* 51, 4 (2018), 1–37. https://doi.org/10.1145/3212695

[2] Miltiadis Allamanis, Earl T. Barr, Soline Ducousso, and Zheng Gao. 2020. Typilus: Neural Type Hints. In *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. ACM, 91–105. https://doi.org/10.1145/3385412.3385997

[3] Varun Bharti, Shashwat Jha, Dhruv Kumar, and Pankaj Jalote. 2025. Automated Type Annotation in Python Using Large Language Models. *arXiv preprint arXiv:2508.00422* (2025).

[4] Varun Bharti, Shashwat Jha, Dhruv Kumar, and Pankaj Jalote. 2025. Automated Type Annotation in Python Using Large Language Models. *arXiv preprint arXiv:2508.00422* (2025). https://arxiv.org/abs/2508.00422

[5] Christopher M Bishop and Nasser M Nasrabadi. 2006. *Pattern recognition and machine learning*. Vol. 4. Springer.

[6] Gilad Bracha and David Griswold. 1993. Strongtalk: typechecking Smalltalk in a production environment. In *Proceedings of the Eighth Annual Conference on Object-Oriented Programming Systems, Languages, and Applications (OOPSLA '93)*. Association for Computing Machinery, New York, NY, USA, 215–230. https://doi.org/10.1145/165854.165893

[7] John Peter Campora, Sheng Chen, Martin Erwig, and Eric Walkingshaw. 2017. Migrating gradual types. *Proceedings of the ACM on Programming Languages* 2, POPL (2017), 1–29.

[8] Robert Cartwright and Mike Fagan. 1991. Soft typing. In *Proceedings of the ACM SIGPLAN 1991 Conference on Programming Language Design and Implementation (PLDI '91)*. Association for Computing Machinery, New York, NY, USA, 278–292. https://doi.org/10.1145/113445.113469

[9] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Neel Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Benjamin Ryder, Michael Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William H. Guss, Alex Nichol, Maria Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Brandon Houghton, Reiichiro Nakano, Prafulla Dhariwal, Ted Chu, Pam Chen, Sam McCandlish Bavarian, Jared Kaplan, and Dario Amodei. 2021. Evaluating large language models trained on code. In *Advances in Neural Information Processing Systems (NeurIPS)*. https://arxiv.org/abs/2107.03374

[10] Kevin Clark, Urvashi Khandelwal, Omer Levy, and Christopher D Manning. 2019. What does bert look at? an analysis of bert's attention. *arXiv preprint arXiv:1906.04341* (2019).

[11] Luis Damas and Robin Milner. 1982. Principal type-schemes for functional programs. In *ACM Symp. on Principles of Programming Languages*. 207–212. https://doi.org/10.1145/582153.582176

[12] Honghua Dong, Jiacheng Yang, Xun Deng, Yuhe Jiang, Gennady Pekhimenko, Fan Long, and Xujie Si. 2025. TypyBench: Evaluating LLM Type Inference for Untyped Python Repositories. *arXiv preprint arXiv:2507.22086* (2025). https://arxiv.org/abs/2507.22086 Under submission; introduces TypeSim and TypeCheck metrics for repository-wide evaluation of LLM type inference.

[13] Michael Furr, Jong-hoon An, Jeffrey S Foster, and Michael Hicks. 2009. Static type inference for Ruby. In *Proceedings of the 2009 ACM symposium on Applied Computing*. 1859–1866.

[14] Michael Furr, Jong hoon An, Jeffrey S. Foster, and Michael Hicks. 2009. Static Type Inference for Ruby. In *Proceedings of the ACM Symposium on Applied Computing (SAC)*. ACM, 1859–1866. https://doi.org/10.1145/1529282.1529700

[15] Yimeng Guo, Zhifei Chen, Lin Chen, Wenjie Xu, Yanhui Li, Yuming Zhou, and Baowen Xu. 2024. Generating Python Type Annotations from Type Inference: How Far Are We? *ACM Transactions on Software Engineering and Methodology* 33, 5 (2024), 123:1–123:38. https://doi.org/10.1145/3652153

[16] Jesse Michael Han, Kexin Pei, Junfeng Yang, and Suman Jana. 2021. TypeBERT: A Transformer-based Model for Type Inference. In *Proceedings of the 44th International Conference on Software Engineering (ICSE)*. ACM, 1290–1301. https://doi.org/10.1145/3468264.3468564

[17] Vincent J. Hellendoorn, Christian Bird, Earl T. Barr, Miltiadis Allamanis, and Premkumar Devanbu. 2018. Deep Learning for Type Inference in Dynamically Typed Languages. In *Proceedings of the 2018 ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*. ACM, 152–162. https://doi.org/10.1145/3236024.3236051

[18] Fritz Henglein. 1994. Dynamic typing: Syntax and proof theory. *Science of Computer Programming* 22, 3 (1994), 197–230.

[19] Simon Holm Jensen, Anders Møller, and Peter Thiemann. 2009. Type analysis for JavaScript. In *International Static Analysis Symposium*. Springer, 238–255.

[20] Simon Holm Jensen, Anders Møller, and Peter Thiemann. 2009. Type Analysis for JavaScript. In *Proceedings of the International Static Analysis Symposium (SAS)*. Springer, 238–255. https://doi.org/10.1007/978-3-642-03237-0_18

[21] Yuhe Jiang, Xun Deng, Jiacheng Yang, Honghua Dong, Gennady Pekhimenko, Fan Long, and Xujie Si. 2025. TypyBench: Evaluating LLM Type Inference for Untyped Python Repositories. In *ICLR 2025 Third Workshop on Deep Learning for Code*. https://openreview.net/forum?id=RDCij13DqN

[22] Milod Kazerounian, Jeffrey S Foster, and Bonan Min. 2021. SimTyper: sound type inference for Ruby using type equality prediction. *Proceedings of the ACM on Programming Languages* 5, OOPSLA (2021), 1–27.

[23] Milod Kazerounian, Jeffrey S. Foster, and Bonan Min. 2021. SimTyper: Sound Type Inference for Ruby using Type Equality Prediction. *Proceedings of the ACM on Programming Languages (OOPSLA)* 5, OOPSLA (2021), 1–27. https://doi.org/10.1145/3485513

[24] Ondřej Lhoták and Philip Wadler. 2025. A simple blame calculus for explicit nulls. *Journal of Functional Programming* 34 (2025), e15.

[25] Zeina Mohamed Magdy Abdelmigeed Mahmoud. 2023. *Theory and Applications for Gradual Type Migration*. University of California, Los Angeles.

[26] Muhammad Zohaib Malik, Veselin Raychev, Predrag Radulovic, and Martin Vechev. 2019. NL2Type: Inferring JavaScript Function Types from Natural Language Information. In *Proceedings of the ACM on Programming Languages (OOPSLA)*, Vol. 3. 1–27. https://doi.org/10.1145/3360571

[27] Robin Milner. 1978. A theory of type polymorphism in programming. *J. Comput. System Sci.* 17 (1978), 348–375.

[28] Amir M Mir, Evaldas Latoškinas, Sebastian Proksch, and Georgios Gousios. 2022. Type4Py: practical deep similarity learning-based type inference for python. In *Proceedings of the 44th International Conference on Software Engineering*. 2241–2252.

[29] Amir M. Mir, Evaldas Latoškinas, and Georgios Gousios. 2021. ManyTypes4Py: A Benchmark Python Dataset for Machine Learning-based Type Inference. In *Proceedings of the 18th IEEE/ACM International Conference on Mining Software Repositories (MSR)*. IEEE/ACM, 585–589. https://doi.org/10.1109/MSR52588.2021.00079

[30] Abel Nieto, Marianna Rapoport, Gregor Richards, and Ondřej Lhoták. 2020. Blame for null. In *34th European Conference on Object-Oriented Programming (ECOOP 2020)*. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 3–1.

[31] Mingyuan Peng, Michael Pradel, and Koushik Sen. 2022. Static analysis with probabilistic models. In *Proceedings of the ACM on Programming Languages (OOPSLA)*, Vol. 6. ACM, 1–27. https://doi.org/10.1145/3563303

[32] Mingyuan Peng, Michael Pradel, and Koushik Sen. 2023. TypeGen: Generating Type Annotations for Python at Scale. In *Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. ACM, 214–226. https://doi.org/10.1109/ASE56229.2023.00025

[33] Yun Peng, Cuiyun Gao, Zongjie Li, Bowei Gao, David Lo, Qirun Zhang, and Michael Lyu. 2022. Static Inference Meets Deep Learning: A Hybrid Type Inference Approach for Python. In *Proceedings of the 44th International Conference on Software Engineering (ICSE '22)*. Association for Computing Machinery, New York, NY, USA, 2019–2030. https://doi.org/10.1145/3510003.3510038

[34] Luna Phipps-Costin, Carolyn Jane Anderson, Michael Greenberg, and Arjun Guha. 2021. Solver-based gradual type migration. *Proceedings of the ACM on Programming Languages* 5, OOPSLA (2021), 1–27.

[35] Benjamin C. Pierce. 2002. *Types and programming languages*. MIT Press.

[36] Michael Pradel, Georgios Gousios, Jason Liu, and Satish Chandra. 2020. TypeWriter: Neural Type Prediction with Search-based Validation. (2020). arXiv:cs.SE/1912.03768

[37] Jeremy G. Siek and Walid Taha. 2006. Gradual Typing for Functional Languages. In *In Scheme and Functional Programming Workshop*. 81–92.

[38] Jeremy G Siek, Michael M Vitousek, Matteo Cimini, and John Tang Boyland. 2015. Refined criteria for gradual typing. In *LIPIcs-Leibniz International Proceedings in Informatics*, Vol. 32. Schloss Dagstuhl-Leibniz-Zentrum fuer Informatik.

[39] Yi Tay, Mostafa Dehghani, Dara Bahri, and Donald Metzler. 2022. Efficient Transformers: A Survey. (2022). arXiv:cs.LG/2009.06732 https://arxiv.org/abs/2009.06732

[40] The Pytype Team. 2023. Pytype: A Static Type Checker for Python. (2023). https://github.com/google/pytype

[41] Satish Thatte. 1989. Quasi-static typing. In *Proceedings of the 17th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL '90)*. Association for Computing Machinery, New York, NY, USA, 367–381. https://doi.org/10.1145/96709.96747

[42] Sam Tobin-Hochstadt and Matthias Felleisen. 2006. Interlanguage Migration: From Scripts to Programs. In *Companion to the 21st ACM SIGPLAN Symposium on Object-oriented Programming Systems, Languages, and Applications (OOPSLA '06)*. ACM, New York, NY, USA, 964–974. https://doi.org/10.1145/1176617.1176755

[43] Andrey Vagin, Vitaly Romanov, and Vladimir Ivanov. 2024. Evaluating Baselines for Type Inference: Static Code Analysis Versus Large Language Model. In *Intelligent Systems Design and Applications*, Ajith Abraham, Anu Bajaj, Thomas Hanne, and Patrick Siarry (Eds.). Springer Nature Switzerland, Cham, 435–444.

[44] Guido van Rossum, Jukka Lehtosalo, and Łukasz Langa. 2014. PEP 484 – Type Hints. https://peps.python.org/pep-0484/. (2014). Accessed: 2025-09-02.

[45] Guido van Rossum, Jukka Lehtosalo, and Łukasz Langa. 2014. PEP 484: Type Hints. Python Enhancement Proposal. (2014). https://peps.python.org/pep-0484/

[46] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017).

[47] Jun Wei, Fei Wang, Xujie Si, Jianbo Chen, Jie Tang, Hanjun Dai, and Le Song. 2020. LambdaNet: Probabilistic Type Inference using Graph Neural Networks. In *Advances in Neural Information Processing Systems (NeurIPS)*. 10636–10647.

[48] Xinyun Wei, Shuo Ren, Ming Zhu, Pengcheng Yin, and Graham Neubig. 2023. TypeT5: Seq2Seq Type Inference with Static Analysis Signals. In *Proceedings of the 37th AAAI Conference on Artificial Intelligence (AAAI)*. AAAI Press, 5265–5273. https://doi.org/10.1609/aaai.v37i4.25587

[49] Cheng Xu, Shuhao Guan, Derek Greene, M Kechadi, et al. 2024. Benchmark data contamination of large language models: A survey. *arXiv preprint arXiv:2406.04244* (2024).

[50] Cheng Xu, Shuhao Guan, Derek Greene, M. Kechadi, et al. 2025. Beyond Memorization: Evaluating the True Type Inference Capabilities of LLMs for Java Code Snippets. *arXiv preprint arXiv:2503.04076* (2025). https://arxiv.org/abs/2503.04076

[51] Frank F Xu, Ziyi Li, Sen Deng, Pengcheng Yin, Graham Neubig, and Vincent J Hellendoorn. 2023. A systematic evaluation of large language models of code. *arXiv preprint arXiv:2308.14322* (2023). https://arxiv.org/abs/2308.14322

[52] Zhaopeng Xu, Yan Xiao, Ting Liu, and Qun Liu. 2016. Predicting Program Properties from "Big Code". In *Proceedings of the 2016 ACM SIGPLAN International Conference on Programming Language Design and Implementation (PLDI)*. ACM, 111–124. https://doi.org/10.1145/2908080.2908113

[53] Michael Yee and Arjun Guha. 2023. Do Machine Learning Models Produce TypeScript Types That Type Check?. In *37th European Conference on Object-Oriented Programming (ECOOP 2023) (Leibniz International Proceedings in Informatics (LIPIcs))*, Vol. 263. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 37:1–37:28. https://doi.org/10.4230/LIPIcs.ECOOP.2023.37

[54] Zhongxing Yu, Chenggang Bai, Lionel Seinturier, and Martin Monperrus. 2019. Characterizing the usage, evolution and impact of java annotations in practice. *IEEE Transactions on Software Engineering* 47, 5 (2019), 969–986.

[55] Kamil Zieliński, Sebastian Bartos, and Andrzej Wąsowski. 2022. The Evolution of Type Annotations in Python: An Empirical Study. In *Proceedings of the 44th International Conference on Software Engineering (ICSE)*. ACM, 1591–1602. https://doi.org/10.1145/3510003.3510079